The other detail we would like to highlight is that we need to "wrap around the array", because we have to add the cost of the edge between the last and first vertices in the sequence (the edge that closes the tour). This can be handled in many ways, using modulo being the most succinct, while the most efficient would be treating this last edge as a separate case outside the `for` loop.

Listing 17.2 Cost Function for TSP

```
function tspTourCost(graph,
P)                                                    #1
  cost ← 0
  for k in {0..|P|} do
                                      #2
    cost ← cost + graph.edgeBetween(P[k],
P[(k+1)%|P|]).weight                  #3
  return cost
```

#1 Method `tspTourCost` takes a graph and a candidate solution `P` (a point in the problem space, i.e. a permutation of the list of vertices in the graph), and computes the cost of the solution as the sum of the weight of all edges in the candidate tour. It's assumed the graph has an edge between every pair of vertices (or, alternatively, that the methods return a special value, like `inf`, if there isn't one).

#2 Cycle through the sequence.

#3 Check the edge between k-th vertex in the sequence and its successor: the one at position `(k+1)` modulo the length of the sequence (so that, when it gets to the last element